Part A

Ex: 5

Title: Urban planning of water supply networks

Problem Description: Design an optimized water distribution network that minimizes the total cost of the infrastructure while ensuring reliable water supply to all areas in the given region.

Method: Represent the problem in the form of a graph. Use the greedy Technique for a spanning tree which has a total minimum weight.

Theory Reference: Module 3

Algorithm

ALGORITHM *Prim(G)*

//Prim's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph $G = \langle V, E \rangle$
//Output: E_T, the set of edges composing a minimum spanning tree of G
$V_T \leftarrow \{v_0\}$ //the set of tree vertices can be initialized with any vertex
$E_T \leftarrow \varnothing$
for $i \leftarrow 1$ **to** $|V| - 1$ **do**
  find a minimum-weight edge $e^* = (v^*, u^*)$ among all the edges (v, u)
  such that v is in V_T and u is in $V - V_T$
  $V_T \leftarrow V_T \cup \{u^*\}$
  $E_T \leftarrow E_T \cup \{e^*\}$
return E_T

Program

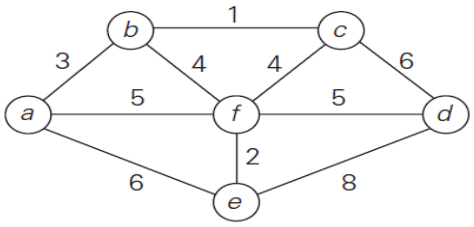
```
import java.util.Scanner;

public class MstPrim

{

   static void prim(int cost[][],int n, int source)

   {

      int[] tvertex = new int[n];

      int a=0,b=0,min,mincost = 0,ne = 0;
```

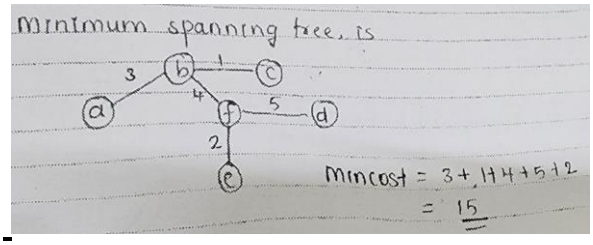
```
//start from the source vertex
tvertex[source] = 1;
while(ne < n-1)
{
   //Find the nearest neighbour
   min = Integer.MAX_VALUE;
   for (int i = 0; i<n; i++)
   {
      if(tvertex[i]==1)
      {
         for(int j = 0;j <n; j++)
         {
          if(cost[i][j] < min && tvertex[j]==0)
          {
               min = cost[i][j];
               a = i;
               b = j;
            }
         }
      }
   }
   //Include nearest neighbour 'b' into MST
```

```
        System.out.println("Pipeline from city " + a +" to city "+ b +" and the cost "+ min +" units ");

        tvertex[b] = 1;

        ne++;

        mincost += min;

      }

     System.out.println("Minimum cost of urban water supply networks "+mincost +" units ");

   }

   public static void main(String[] args)

   {

      Scanner s = new Scanner(System.in);

      System.out.println("Enter the no. of Cities");

      int n = s.nextInt();

      int[][] c = new int[n][n];

      System.out.println("Enter the cost of laying pipelines between the two cities");

      for(int i=0;i<n;i++)

      {

         for(int j=0;j<n;j++)

         {

            c[i][j]=s.nextInt();

            if (c[i][j] == 0)

                        c[i][j] = Integer.MAX_VALUE;
```

```
        }
    }
    System.out.println("Enter the source city");
    int source=s.nextInt();
    prim(c,n,source);
  }
}
```

Output






```
run:
Enter the no. of Cities
6
Enter the cost of laying pipelines between the two cities
0 3 0 0 6 5
3 0 1  0 0 4
0 1 0 6 0 4
0 0 6 0 8 5
6 0 0 8 0 2
5 4 4 5 2 0
Enter the source city
0
Pipeline from city 0 to city 1 and the cost 3 units
Pipeline from city 1 to city 2 and the cost 1 units
Pipeline from city 1 to city 5 and the cost 4 units
Pipeline from city 5 to city 4 and the cost 2 units
Pipeline from city 5 to city 3 and the cost 5 units
Minimum cost of urban water supply networks 15 units
BUILD SUCCESSFUL (total time: 1 minute 10 seconds)
```